PACMAN TECHNICAL WHITEPAPER & ARCHITECTURE

This document contains the deep technical details, security model, and CLI documentation for Pacman, the autonomous Hedera DeFi agent. For a high-level overview, see the main `*README.md*` *.*

How It Works

Four ways to interact with Pacman — all driving the same core engine, all settling on Hedera:

```
graph TD
    %% Styling %%
    classDef interface fill:#1e1e2e,stroke:#cba6f7,stroke-
width:2px,color:#fff,rx:8,ry:8
    classDef ai fill:#ff2052,stroke:#fff,stroke-width:2px,color:#fff,rx:8,ry:8
    classDef brain fill:#181825,stroke:#ff2052,stroke-dasharray: 5
5,color:#fff,rx:8,ry:8
    classDef engine fill:#313244,stroke:#a6e3a1,stroke-
width:2px,color:#fff,rx:8,ry:8
    classDef hedera fill:#000000,stroke:#00a8e8,stroke-
width:3px,color:#fff,rx:8,ry:8
    classDef ext fill:#11111b,stroke:#f9e2af,stroke-width:2px,color:#fff,rx:8,ry:8

    %% Users & Interfaces %%
    subgraph UI ["📱 User Interfaces & Touchpoints"]
        Telegram["Telegram App<br/>(Conversational UI)"]:::interface
        Dashboard["Web Dashboard<br/>(Visual Portfolio)"]:::interface
        HumanCLI["Terminal CLI<br/>(Power Users)"]:::interface
    end

    %% OpenClaw AI %%
    subgraph Intelligence ["🧠 OpenClaw AI Middleware"]
        Agent["🤖 OpenClaw Agent<br/>(Natural Language → Actions)"]:::ai
        Context["📝 SKILL.md<br/>(Hedera Logic / Constraints)"]:::brain
        Context -.->|Rules & Context| Agent
        Telegram ===|Chat / Webhooks| Agent
    end

    %% Core Application %%
    subgraph Pacman ["⚡ Pacman Core Engine (Local Machine)"]
        Controller["Core Controller<br/>(Deterministic Backend)"]:::engine
        Router["SaucerSwap V2 Router<br/>(Price Pathfinding)"]:::engine
        Exec["TX Broadcaster<br/>(Key Mgmt & Execution)"]:::engine
        Plugins["Plugin System<br/>(Autonomous Daemons)"]:::ext

        Agent ===|Executes CLI subcommands| HumanCLI
        HumanCLI -->|Triggers| Controller
        Dashboard -.->|Read-Only Status| Controller

        Controller --> Router
        Controller --> Exec
```

```
    Controller --> Plugins
end

%% Plugins %%
subgraph Addons ["🧩 Active Plugins"]
    PL["Power Law Rebalancer<br/>(Auto Trading)"]:::ext
    HCS10["HCS-10 Protocol<br/>(Agent Messaging)"]:::ext
    Plugins --> PL
    Plugins --> HCS10
end

%% Hedera %%
subgraph Blockchain ["🌐 Hedera Network (On-Chain Settlement)"]
    SSV2["SaucerSwap V2<br/>(DEX Smart Contracts)"]:::hedera
    HCS["Hedera Consensus Service<br/>(Feedback / Logs)"]:::hedera
    HTS["Hedera Token Service<br/>(Transfers / Approvals)"]:::hedera

    Router ===>|Quote/Swap| SSV2
    Exec ===>|Broadcast| HTS
    Exec ===>|Emit Events| HCS
    HCS10 ===>|Publish/Subscribe| HCS
end
```

Interface	What It Does	Status
OpenClaw Agent	Natural language wallet — swap, send, check balances, manage portfolio via chat. Reads `SKILL.md` for decision-making.	Live
Telegram	Button-driven wizards for common flows (<200ms, no LLM round-trip). Natural language falls through to the agent.	Live
CLI	The deterministic command layer. 30+ commands that both humans and AI agents operate identically.	Live
Dashboard	Local web UI for portfolio monitoring and system health. Read-only for now.	Live (view-only)

Technical Features Deep-Dive

Talk to Hedera Through OpenClaw (Telegram)

Capability	How
Swap tokens	`swap 10 HBAR for USDC` — natural language or button wizards
Send transfers	`send 50 USDC to 0.0.xxx` — whitelisted destinations only
Check portfolio	`balance` — all holdings with live USD values
Set limit orders	`order buy HBAR at 0.08 size 100` — background daemon
Stake HBAR	`stake` — to consensus nodes

Manage liquidity	`lp` — V2 pool positions

Direct SaucerSwap V2 Integration

We built an open source connection to SaucerSwap V2 from scratch — the first available to the Hedera community.

Capability	Detail
Token types	HTS tokens, native HBAR, EVM tokens — swaps across all of them
Fee tiers	Three V2 fee tiers with automatic selection
Hub routing	Routes through USDC and HBAR liquidity hubs
HBAR/WHBAR	Automatic wrapping/unwrapping (WHBAR is never user-facing)
Pool validation	Depth checks before execution, stale data detection

HCS Agent Feedback System

Every Pacman instance can post bugs, suggestions, and feedback directly to a shared HCS topic — no GitHub account required. This creates an automated improvement loop: usage generates bug reports, reports get timestamped on-chain, developers (or coding agents) pick them up and push fixes.

Plugin Architecture

Build your own strategies, tools, and daemons without touching core wallet code. Extend `BasePlugin`, drop it in `src/plugins/`, and you have direct blockchain access.

Plugin	What It Does
Power Law Rebalancer	Autonomous BTC/USDC rebalancing daemon
Telegram Wallet Bot	Standalone polling bot with button wizards
HCS Publisher	Daily signal broadcasting to Consensus Service
HCS-10	Cross-agent communication protocol
Discord Bot	Portfolio monitoring via Discord
Limit Order Engine	Background price polling and passive swap execution

Training Data Pipeline

Every command automatically generates structured fine-tuning data — from your first interaction.

- `agent_interactions.jsonl` : Raw operational log (command, output, timing, errors)
- `instruction_pairs.jsonl` : OpenAI-compatible SFT pairs
- `live_executions.jsonl` : Detailed tx telemetry (gas, rates, hashes, routes)

Security Model

The key design decision: **your private keys never touch an AI API.**

The agent runs commands — it doesn't see your keys, it doesn't read the plumbing, it doesn't know how transactions are constructed. It selects an action (`swap 5 HBAR for USDC`), the CLI executes it deterministically, and returns the result. Software handles the crypto. AI handles the intent.

Layer	How It Works
Key isolation	Keys live in `.env` on your machine. XOR-obfuscated in memory. Never transmitted.
Deterministic execution	Every command produces the same result for the same input. No probabilistic failures.
Transfer whitelists	All outbound transfers blocked unless destination is explicitly approved. EVM addresses blocked entirely.
Safety limits	`governance.json` enforces: $100 max per swap, $100 daily, 5% slippage cap, 5 HBAR gas reserve.
Agent guardrails	Agent can't modify its own limits, can't bypass whitelists, can't access keys.

Hedera Services Used

Service	How We Use It
HTS	Token creation, association, transfers, ERC20 approvals via precompile
HCS	Signal broadcasting, cross-agent feedback (HCS-10), timestamped messaging
EVM	SaucerSwap V2 router/quoter, multicall, exact-in/exact-out swaps
Mirror Node	Balances, tx history, pool data, EVM alias resolution, NFT metadata
Accounts	Multi-account management, independent ECDSA keys, nickname discovery

---

CLI Commands

The CLI is the deterministic command layer that both humans and AI agents operate. Every function the agent can perform, you can run directly.

```
TRADING        swap, swap-v1, price, slippage
PORTFOLIO      balance, status, history, tokens, nfts
TRANSFERS      send, receive, whitelist
ACCOUNT        account, associate, setup, fund, backup-keys
STAKING        stake, unstake
LIQUIDITY      lp, pool-deposit, pool-withdraw, pools
LIMIT ORDERS   order buy/sell/list/cancel/on/off
ROBOT          robot signal/status/start/stop
MESSAGING      hcs, hcs10, feedback
SYSTEM         doctor, refresh, logs, docs, help
```

Run `./launch.sh help` for the full list.

Repository Structure

```
cli/              Command handlers (30+ commands, modular sub-commands)
  commands/       Swap, balance, orders, wallet, staking, HCS, NFTs, etc.
src/              Core engine
  controller.py   SDK facade — the only thing CLI talks to
  router.py       V2 pathfinding with cost-aware hub routing
  executor.py     Transaction broadcaster (swaps, approvals, transfers)
  plugins/        Plugin system (Power Law, Telegram, HCS, Discord, etc.)
lib/              Integrations (SaucerSwap, Telegram, Discord, prices)
data/             Config, pools, governance, tokens, ABIs
openclaw/         AI agent workspace (SKILL.md, persona, decision trees)
dashboard/        Local web monitoring (read-only)
scripts/          Utilities, data harvesting, pool refresh
tests/            Test suites
```